

Результат месяца: путь, траектория и поведение

M16 • 21.01.2028 - 15.02.2028 • 75 часов

Ты подготовишь сравнительные испытания планировщиков и небольшую автономную задачу с деревом поведения. Робот получит цель, найдёт допустимый путь, построит по нему траекторию с ограничением скорости и будет отслеживать выполнение, явно реагируя на невозможность продолжить. Примеры месяца — маршрут корпуса среди препятствий и подход манипулятора к объекту. Эти основы нужны до перехода к планированию постановки стоп.

Три разных вопроса

Поиск отвечает, какие состояния соединяют старт и цель. Геометрический путь $q(s)$ описывает последовательность конфигураций без времени. Траектория $q(t)$ добавляет время, скорость и ускорение. Логика задачи решает, когда планировать, ожидать, отменять и восстанавливаться. Успех A^* не означает, что привод сможет выполнить путь; успешное планирование руки не означает, что предмет уже захвачен.

Что использовать из предыдущих месяцев

Возьми карту с неизвестными клетками из M15, геометрию сцен из M14, кинематику звена и ограничения приводов из ранних модулей. Проверь, что старт и цель выражены в одной системе, а единицы углов и линейных размеров согласованы. Для входа достаточно уметь читать URDF, запускать ROS 2 actions, строить граф и объяснять производную. Повтори на одной траектории, как из угла получить рад/с и рад/с².

Два больших стенда

Стенд А — 2D карта помещения и корпус с конечной площадью проекции на плоскость (footprint). Для знакомства с Nav2 можно использовать колёсную базу в симуляции. Стенд В — готовая модель манипулятора в MoveIt 2 со столом и коробкой. Возьми по одному подготовленному пакету, совместимому с Ubuntu 24.04 и ROS 2 Jazzy. В этом месяце не создавай с нуля сложную модель робота и новое окружение.

Итоговый комплект содержит собственные Dijkstra/ A^* , сравнение минимум двух методов, опыт с выборками конфигураций, проверку траекторий, журналы BT и FSM и сценарии восстановления. Планирование опор, кинодинамика гексапода и оптимизация контактов относятся к дальнейшей специализации. Уже сейчас объясни, почему круглая проекция корпуса и свободная клетка пола ещё не подтверждают возможность конкретного шага многоногого робота.

75 часов: от карты к управляемому выполнению

Четыре блока работы

1–20: графы, BFS, Dijkstra, A*, эвристики и карта стоимости; 10 часов теории и анализа +10 практики. 21–40: пространство конфигураций, проверка столкновений, PRM/RRT/RRT* и сцена MoveIt; 10+10. 41–60: временная параметризация, сплайны, ограничения скорости, ускорения и рывка; 10+10. 61–75: FSM, BT, восстановление, таймауты и основы MDP/POMDP; 5+10. Общий бюджет — 35 часов разбора и 40 часов практики.

Для первых трёх теоретических частей используй занятия по 2 часа; последнюю разбей на 2+2+1. Практические воскресные блоки по 10 часов включают реализацию, измерение и отчёт; перерывы не считаются часами учёбы. Нумерация ниже задаёт последовательность учебных часов, а не календарные даты. Граничные даты модулей могут совпадать; отпуска, отдых и отдельные контрольные этапы сохраняются в исходном календаре без повторного учёта часов.

Структура репозитория

В `maps/` храни синтетические и входные карты; в `search/` — алгоритмы и общий интерфейс; в `sampling/` — настройки OMPL; в `scenes/` — объекты MoveIt и параметры робота; в `trajectories/` — файлы `q,t` и пределы; в `autonomy/` — дерево, FSM и адаптеры; в `benchmark/` — сценарии, значения `seed` и результаты. Каждый результат связывай с версией карты и проекцией корпуса. После перемещения препятствия нельзя незаметно продолжать работу по старой геометрии.

Правило одной гипотезы

В опыте «A* расширяет меньше вершин» оставь одинаковые граф, стоимость и порядок разрешения равенств. В опыте «RRT* улучшает путь» сравни время и стоимость при том же бюджете, включая неудачные запуски с разными `seed`. В опыте «сглаживание улучшает движение» повторно проверь столкновения: сплайн может срезать угол через стену. Сначала сформулируй причину сравнения, затем выбери метрику.

Работа с GUI заканчивается сохранением машинно-читаемой сцены и команд запуска. Не ограничивайся ручным перетаскиванием цели в RViz. Для каждой задачи подготовь небольшой контрольный пример, который другой человек может проверить без полного стенда: известный оптимальный путь, отрезок через препятствие, заведомо слишком быструю траекторию или ожидаемую трассу состояний.

Поиск на графе и стоимость маршрута корпуса

Что изучить

BFS ищет кратчайший путь по числу рёбер одинаковой стоимости; Dijkstra работает с неотрицательной стоимостью; A* добавляет эвристическую оценку h до цели. Для оптимальности изучи допустимость и согласованность h , переоткрытие вершин и критерий завершения. На 4-связной решётке с единичным шагом подходит манхэттенское расстояние; при других движениях и ценах его допустимость нужно проверить заново.

Карта стоимости (costmap) хранит назначенную цену движения, а карта занятости — сведения о занятых, свободных и неизвестных клетках. Для корпуса конечного размера учти проекцию корпуса (footprint) либо расширь препятствия на радиус круговой модели с обоснованным запасом. Зазор должен учитывать геометрию и неопределённость. Явно задай обращение с unknown: отсутствие наблюдений не разрешает движение.

Задача 1. Dijkstra и A* на карте робота

Создай карты: открытое поле, U-образная ловушка и два коридора с разными ценами. Реализуй общий интерфейс `solve(start, goal, map)`, восстановление пути и счётчик расширенных вершин. Выход: путь, стоимость, время работы `runtime`, число раскрытий `expansions` и статус `no_path`. Проверь `start=goal`, цель в препятствии, недостижимую цель и одинаковую стоимость A*/Dijkstra при допустимой h . BFS используй как контроль только для одинаковой цены рёбер.

Задача 2. Узкий проход и динамическое препятствие

Возьми круговую модель корпуса и проход вокруг коробки. Построй две карты стоимости для разных размеров проекции корпуса и сравни доступность прохода. После построения пути добавь препятствие посередине маршрута. Пометь прежний путь устаревшим, останови условное выполнение и перепланируй движение. Сохрани версии карты и плана, журнал признания плана недействительным и новый путь либо корректный отказ.

Проверка: каждый сегмент проверяется по актуальной карте; диагональ не проходит между касающимися углами двух занятых клеток. Сохрани случай, когда центр корпуса проходит, но проекция корпуса задевает стену. Для ноги объясни будущую разницу: маршрут центра корпуса не задаёт последовательность контактов, а неровность по высоте требует другой модели состояния. В M16 достаточно показать этот предел и не называть 2D путь готовым планом ходьбы.

Поиск по выборкам и сцена манипулятора

Конфигурационное пространство

Конфигурация q содержит суставные углы, положение корпуса либо другие координаты системы. Препятствие в рабочем пространстве превращается в множество запрещённых q . Проверка одной конечной точки не гарантирует допустимость движения между точками. Разбери ограничения суставов, самостолкновения, столкновение с окружением и прикреплённым предметом. Метрика в пространстве конфигураций (C-space) должна учитывать единицы и периодические углы.

PRM строит сеть конфигураций для повторных запросов; RRT расширяет дерево; RRT* улучшает связи для уменьшения стоимости при росте выборки. Вероятностная полнота и асимптотическая оптимальность — свойства при допущениях и предельном бюджете, а не гарантия решения за две секунды. Узкий проход может редко попадать в выборку. Упрощение пути после поиска тоже меняет результат и требует проверки.

Задача 3. Сравнение поиска по выборкам

Начни с 2D модели точки или диска среди прямоугольных препятствий в OMPL. Сравни RRTConnect с RRT* либо PRM при одинаковом времени на запрос: минимум 10 начальных значений генератора случайных чисел (seed) для трёх сцен. Сохрани долю точных решений, время первого решения, длину и минимальный зазор; затраты на построение PRM учти отдельно. Подготовь настройки configs, список seed-list и таблицу всех запусков, включая таймауты. Отсутствующее решение не учитывай как путь нулевой длины.

Задача 4. Сцена планирования MoveIt

Возьми готовую руку, стол и коробку. Задай размеры, положение и ориентацию объектов в одной системе координат; укажи старт и цель и спланируй подход в симуляции. Затем прикрепи объект к захвату и повтори расчёт. Сохрани сцену, допустимый путь и пример отказа. Проверь, что прикреплённый предмет участвует в проверке столкновений, а его система координат и масштаб заданы верно.

Отдельно проверь путь с достаточно мелкой дискретизацией и сравни с грубой: тонкое препятствие между опорными точками пути должно быть обнаружено. Документируй ограничение дискретной проверки; без непрерывной проверки столкновений это не математическое доказательство отсутствия контакта на всём непрерывном пути. Будущая рука использует этот стенд для подхода к объекту, а будущая нога — похожую проверку пространства движения звеньев.

Траектория: время, гладкость и пределы приводов

Основные разделы

Разбери линейную интерполяцию, кубический и квинтический профиль, непрерывность положения, скорости и ускорения. Рывок (jerk) — производная ускорения: для вращательного сустава рад/с³. Геометрический сплайн может быть гладким по параметру s , но после неверной временной параметризации стать недопустимым. Даже при допустимых скоростях траектория может требовать слишком большого момента; динамические ограничения нужно явно отделять.

Для простого движения от q_0 к q_1 используй $q(t)=q_0+(q_1-q_0)\cdot s(t/T)$, где $s(\tau)=10\tau^3-15\tau^4+6\tau^5$, $0\leq\tau\leq 1$. В начале и конце скорость и ускорение равны нулю. Разбери производные: скорость масштабируется как $1/T$, ускорение как $1/T^2$, рывок — как $1/T^3$. Это даёт понятный способ замедлить учебное движение, но не проверяет столкновения и динамику автоматически.

Задача 5. Профиль сустава ноги или пальца

Задай q_0, q_1 и учебные ограничения в конфигурации, явно подписав их как параметры симуляции. Рассчитай профиль для трёх T , построй q, v, a, j и найди минимальное выбранное T , удовлетворяющее пределам с запасом. Выход: trajectory.csv, графики, допущения и валидатор. Проверь $q_0=q_1$, $T\leq 0$, нечисловые входы и слишком короткое движение. Для реального привода лимиты берутся из паспорта и испытаний нагрузки.

Задача 6. Проверить путь после сглаживания

Возьми путь задачи 4, добавь временную параметризацию доступным средством MoveIt и проверь весь результат: лимиты суставов, времена строго возрастают, производные, столкновения. Для выбранной библиотеки выясни, меняется ли геометрия при обработке. Выход: исходный и обработанный пути, времена и причины отклонения. Сконструируй угол рядом с препятствием, где гладкая интерполяция пересекает его.

При знакомстве с оптимизацией траектории ограничься постановкой: переменные — состояния и управления; ограничения — динамика и препятствия; целевая функция — время, энергия или гладкость. Реши небольшой скалярный пример либо разбери готовый ход решения, не создавая полноценный оптимизатор. Выбор топологии пути и непрерывная оптимизация траектории решают разные части общей задачи; оба результата нужно проверять.

Автономное поведение: состояния и восстановление

Минимальная логика задачи

Определи состояния `idle`, `planning`, `executing`, `waiting`, `recovering`, `failed` и `completed`. Для каждого укажи входы, действия, выходные условия и разрешённую отмену. FSM удобна для явного учёта переходов; дерево поведения (`behavior tree`) — для повторяемых условий и действий со статусами `SUCCESS`, `FAILURE`, `RUNNING`. Изучи `Sequence`, `Fallback`, реактивную проверку условий и прерывание асинхронного действия.

Восстановление должно иметь причину и предел числа попыток. При устаревшей карте обнови наблюдения и перепланируй путь; недостижимую цель бесконечные повторы достижимой не сделают. Отмена активного действия ROS (`action`) требует подтверждения и однозначного состояния, иначе движение может продолжиться после смены цели. Локальные таймауты измеряй монотонным таймером; модельное время и воспроизведение записи описывай отдельно.

Задача 7. Дерево поведения для маршрута

Собери «проверить локализацию → построить путь → следовать пути» с веткой ожидания свежих данных и ограниченным перепланированием. Сначала реализуй детерминированные заглушки действий, затем подключи действие `Nav2` в симуляции. Выход: XML либо код BT, таблица переходов и журнал событий. Проверь `FAILURE` планировщика, непрекращающийся `RUNNING` и отмену пользователем; число попыток ограничено конфигурацией.

Задача 8. Измерение времени восстановления

В симуляторе перекрой путь, затем освободи его; измерь время обнаружения, время до нового плана и время возобновления. Для постоянного препятствия ожидается `failed` либо ожидание оператора, а не ложный `completed`. Выход: запись, настройки BT, события и два графика. Не переноси команды колёсного примера `Nav2` непосредственно на шагающий корпус: для него нужен отдельный исполнитель с проверками контактов.

MDP и POMDP на понятном примере

Опиши выбор между коротким неизвестным участком, обходом и осмотром через состояние, действие, переход и вознаграждение. В POMDP истинное состояние скрыто; решение опирается на распределение гипотез (`belief`), уточняемое по наблюдениям. Составь таблицу двух гипотез о проходимости и обнови вероятность после наблюдения камеры. Обучение с подкреплением здесь не требуется. Неопределённость карты нужно учитывать, прежде чем разрешать движение.

Интеграционный стенд: план имеет срок действия

Требования к запросу и плану

В запрос включи `request_id`, актуальное стартовое положение и ориентацию, цель, `frame_id`, `map_version`, модель робота и пределы. Ответ содержит статус `exact/approximate/no_path/timeout`, путь, условия его допустимости и параметры проверки. Статус `approximate` не подтверждает достижение цели. Перед приёмом траектории исполнитель проверяет версию карты, доступность локализации и актуальность начального состояния.

Задача 9. Связать поиск, траекторию и поведение

1. Проверь входные данные. 2. Рассчитай путь A* либо Movelt для своего стенда. 3. Независимо проверь столкновения. 4. Задай время движения и проверь пределы. 5. Передай траекторию исполнителю в симуляции. 6. Следи за изменениями карты и статуса. 7. При нарушении запрети приём следующего сегмента, отмени действие `action` и переведи ВТ к восстановлению. 8. Сохрани конечное состояние и его причину.

Для маршрута корпуса используй путь между двумя точками карты. Для руки используй подход к положению перед захватом и возврат в исходное состояние; сам цикл захвата будет в M17. Общий валидатор проверяет общие требования к данным, но геометрия у стендов разная. Не представляй одинаковый список точек как универсальный исполнитель для колёс, ноги и манипулятора.

Задача 10. Воспроизвести асинхронную ошибку

Создай ситуацию: план A задержался, пользователь уже запросил B, затем пришёл старый успешный ответ A. Исполнитель должен отвергнуть A по `request_id` и версии, а журнал показать причину. Повтори с задержанным подтверждением `cancel` и завершением старого `action`. Выход: воспроизводимый тест без GUI и последовательность событий. Это пример гонки асинхронных событий в робототехнике: математически корректный путь может относиться к уже неактуальной задаче.

Состав итогового пакета

Сохрани сравнительные испытания в одинаковых условиях со всеми `seed`, сцены Movelt и Nav2, проверки пути и траектории, дерево и таблицу FSM, восемь сценариев отказа и README с командами, версиями и ограничениями. Два коротких видео должны показывать обычное выполнение и корректное восстановление. Время измеряй по журналам событий, а не по смонтированному видео. Этот программный стенд не подтверждает физическую безопасность привода и реального рабочего пространства.

Сценарии, которые обязан выдержать планировщик

Геометрия и данные

S1 — start либо goal внутри препятствия: до поиска возвращается `invalid_request`, выполнение не начинается. S2 — проход уже проекции корпуса: путь точки может существовать, но план корпуса должен быть отклонён. Вручную проверь самое узкое место. S3 — тонкая стенка между опорными точками пути: независимая проверка столкновений должна найти пересечение; сравни два разрешения проверки.

S4 — полоса `unknown` через карту: режим `forbid` даёт обход либо отказ; `allow` допустим лишь как явно обозначенный учебный опыт. Сохрани обе настройки, не меняя их незаметно при неудаче. S5 — новое препятствие во время движения: старый план отменяется, `version` увеличивается, новый план использует актуальную сцену. Измерь время обнаружения и запиши реакцию конечного автомата.

Время и логика

S6 — невыполнимая траектория: сократи длительность до нарушения пределов скорости или ускорения. Проверка должна отклонить результат, указав участок и измеренную величину. Отдельно добавь немонотонные временные метки. S7 — таймаут и зависшее действие: заглушка выдаёт `RUNNING` дольше лимита. Дерево должно ограничить попытки восстановления и завершиться понятным статусом. Запиши попытки и время ожидания, даже если путь впоследствии найден.

S8 — отмена и поздний ответ: запрос B заменяет A, затем приходит результат A. Ни один сегмент A не должен быть принят после подтверждённой смены актуального запроса. Проверь также потерю локализации из M15: новая глобальная цель блокируется, а завершившееся действие не помечается как успешное только из-за прекращения обновлений.

Метрики и правила сравнения

Для поиска измеряй `expansions` — число раскрытых вершин, `cost` — сумму заданных цен рёбер, `runtime` — длительность выбранного участка кода. Для методов с выборками измеряй долю точных решений, время первого решения, итоговую стоимость, минимальный зазор и число проверок столкновений. Таймаут учитывай отдельно; медиану времени успешных запусков показывай вместе с долей успеха.

Для автономной задачи измеряй долю успеха, число восстановлений, время обнаружения нарушения и построения нового плана, число отклонённых старых команд. Укажи часы, частоту вызовов `tick` и оборудование. Длина пути корпуса — сумма длин сегментов в метрах. Для суставного пути сумма норм угловых приращений имеет иной смысл: задай веса суставов и не сравнивай её с метрами. Минимальный зазор (`clearance`) вычисляется между геометрическими телами в проверенных конфигурациях; укажи дискретизацию и единицы. Карта может штрафовать близость стены, поэтому кратчайший путь не всегда имеет наименьшую стоимость. Отсутствие столкновений в конечном наборе симуляций характеризует этот набор и не доказывает их отсутствие при любых данных, параметрах и реальном контакте.

Занятия 1-40: графы и пространство конфигураций

Часы 1-10: пять занятий по 2 часа

1-2: различие пути, траектории и решения задачи; составь схемы входов и выходов. 3-4: BFS/Dijkstra/A*, очереди и стоимость; вручную раскрой маленькую карту. 5-6: допустимость и согласованность эвристики и завершение A*. 7-8: проекция корпуса, расширение препятствий, unknown и единицы карты стоимости. 9-10: протокол сравнения, динамическое препятствие и проверка версий. Используй опыт алгоритмических задач, но добавь реальный размер робота и последствия неизвестных клеток.

Часы 11-20: практикум 1

11-12: задача 1, общий интерфейс и Dijkstra. 13-14: A*, проверка оптимальной стоимости и BFS для единичных рёбер. 15-16: три карты, метрики и отображение раскрытых вершин. 17-18: задача 2, разные проекции корпуса и узкий проход. 19-20: изменение карты, признание прежнего плана недействительным и воспроизводимый `no_path`. К концу блока подготовь базовый метод для сравнения с последующими алгоритмами.

Часы 21-30: пять занятий по 2 часа

21-22: пространство конфигураций двухсуставной руки, самостолкновения и пределы. 23-24: PRM/RRT/RRT*, метрика, выборки и ограничение времени. 25-26: вероятностные свойства и узкие проходы; выпиши условия, недостижимые за конечное время. 27-28: сцена MoveIt, прикреплённые объекты и матрица проверки столкновений. 29-30: независимая проверка пути и дискретизация движения.

Часы 31-40: практикум 2

31-32: задача 3, OMPL и сцены для двух планировщиков. 33-34: 10 значений seed, таймауты и полное сравнение. 35-36: задача 4, готовая рука, стол и коробка. 37-38: перемещение прикреплённого объекта, проверка системы координат и размеров. 39-40: тонкое препятствие, недостижимая цель и отчёт об ограничениях поиска по выборкам. Общее время одинаково, затраты на предварительное построение PRM учитываются отдельно.

Если GUI MoveIt не запускается, запиши ограничения среды и попробуй открыть готовую сцену без графического интерфейса в отведённое время. Не трать весь практикум на другую ОС. При необходимости упрости руку, сохранив пределы суставов, препятствие и проверку прикреплённого объекта. Разбор конкретной ошибки проверки столкновений полезнее добавления новых моделей.

Занятия 41-75: траектории и автономная задача

Часы 41-50: теория по 2 часа

41-42: производные $q(t)$, единицы, линейная и кубическая интерполяция. 43-44: квинтический профиль и масштабирование T . 45-46: скорость, ускорение, рывок, граничные условия и пределы привода. 47-48: временная параметризация MoveIt и повторная проверка геометрии. 49-50: основы оптимизации траектории, целевая функция и ограничения; сравнение с поиском топологии пути.

Часы 51-60: практикум 3

51-52: задача 5, профиль одного сустава и три длительности. 53-54: аналитические и численные производные, валидатор и крайние входы. 55-56: задача 6, временная параметризация пути руки. 57-58: сглаживание рядом с препятствием и независимая проверка. 59-60: файлы траекторий, лимитов и причины отказа; подготовку интерфейса симуляционного исполнителя.

Часы 61-65: теория 2+2+1

61-62: FSM и BT, RUNNING/SUCCESS/FAILURE, Sequence/Fallback. 63-64: отмена, таймаут, восстановление и поздние ответы; нарисуй последовательность событий задачи 10. 65: MDP/POMDP через выбор обхода, осмотра или движения и распределение гипотез о проходимости. Дополнительный курс RL в эти часы не входит: нужно понять, какая информация недоступна простой логике.

Часы 66-75: практикум 4

66-67: задача 7, дерево с детерминированными заглушками действий и проверка переходов. 68-69: задача 8, симуляция Nav2, временное и постоянное препятствия. 70-71: задача 9, требования к передаче плана, траектории и исполнения, возраст карты. 72-73: задача 10, S1-S8, отмена и поздние ответы. 74-75: воспроизведение с чистого запуска, сравнительная таблица и итоговый отчёт. Все задания входят в 75 часов.

Что сокращать первым

Сначала откажись от третьего планировщика, расширенного сравнения способов сглаживания и дополнительных интерфейсов. Затем сократи число сцен до трёх и длительность симуляций, сохраняя все виды отказов. Обязательны Dijkstra/A*, один опыт с выборками, сцена столкновений MoveIt, ограничения траектории и измерение восстановления BT. Проверку устаревшего плана сохраняй: асинхронная гонка может свести на нет правильность алгоритма поиска.

Приёмка: правильный отказ тоже является результатом

Обязательные результаты проверки

Сравни минимум два планировщика по одному протоколу; для Dijkstra/A* проверь стоимость на контрольных графах. В опыте с выборками используй несколько seed, учитывай таймауты и проверяй полученный путь. Сцена MoveIt должна включать препятствие и прикрепленный объект. Невыполнимую траекторию отклоняй с конкретной причиной; после сглаживания повторно проверяй столкновения и пределы.

Дерево поведения выполняет задачу в нормальной сцене, восстанавливается после временного препятствия и завершает попытки при постоянной недостижимости. Отмена работает при RUNNING, поздний результат не заменяет актуальную задачу. Журналы связывают map_version, request_id, план и исполнение. Все восемь сценариев воспроизводятся; инструкция запуска не зависит от ручных действий автора в GUI.

Самопроверка

1. Когда BFS и Dijkstra дают одинаковую стоимость? 2. Как неверная h лишает A* ожидаемой оптимальности? 3. Почему размер проекции корпуса меняет возможность найти путь? 4. Чем PRM отличается по использованию от RRT? 5. Почему RRT* не обещает оптимальный путь за фиксированное время? 6. Что пропускает проверка только опорных точек пути? 7. Как меняются v, a, j при удвоении T ? 8. Почему гладкий сплайн может столкнуться со стеной? 9. Чем ограничения скорости отличаются от ограничений момента? 10. Почему RUNNING не является ни успехом, ни отказом? 11. Как отменить старую задачу и не принять её поздний ответ? 12. Какую дополнительную информацию требует POMDP по сравнению с полностью наблюдаемым состоянием?

Передача к манипуляции и ногам

В M17 передай сцену руки, объекты столкновений, путь к положению перед захватом, проверенный исполнитель, пределы и журнал событий. Для дальнейшей ходьбы сохрани карту с unknown, допущения о проекции корпуса и ограничения, отсутствующие в 2D маршруте. Выбор контактов, устойчивость, достижимость стоп и динамика корпуса потребуют новой модели. Механизм отмены и проверки актуальности задачи останется применимым.

Финальная демонстрация должна включать no_path, слишком быструю траекторию и восстановление, помимо успешного маршрута. Расскажи, какой именно уровень гарантий даёт твой проверочный набор. Статус «не удалось построить допустимый план» полезнее движения по неопределённому результату. При этом не утверждай физическую безопасность реальной системы на основании одного симуляционного месяца.

Литература: короткий маршрут по разделам

Литература преимущественно на английском; план и определения — на русском. Читай выбранные разделы перед задачей, а не книги целиком. Основное чтение входит в существующие 35 часов разбора; справочник открывай по необходимости.

Основное чтение

С. Лаваль, *Planning Algorithms* (2006). [Авторская онлайн-книга](#). Гл. 2 Discrete Planning — поиск на графе; гл. 4 The Configuration Space — препятствия в C-space; гл. 5 Sampling-Based Motion Planning — PRM/RRT. Перед задачами 1–4 выпиши постановку, критерий допустимости и условие завершения.

К. Линч, Ф. Парк, *Modern Robotics* (2017). [Книга и учебные разделы](#). §9.1–9.2 Point-to-Point Trajectories, §9.3 Polynomial Via Point Trajectories, §9.4 Time-Optimal Time Scaling. В задачах 5–6 различай $q(s)$ и $q(t)$, вычисли скорость и ускорение, проверь пределы.

Справочно для управления задачами

М. Колледанкизе, П. Эрген, *Behavior Trees in Robotics and AI: An Introduction*. [Авторская версия книги](#). §1.3 Classical Formulation of BTs, §2.1 Finite State Machines; в гл. 3 Design principles — §3.1–3.2 об условиях успеха и реактивности. Для задач 7–8 нарисуй дерево планирования, следования и восстановления; проверь возврат RUNNING, остановку и повторную оценку условий.

Три рабочих справочника

[OMPL: How to Benchmark Planners](#) — Writing benchmarking code и Processing the benchmark log. В задаче 3 сохраняй exact/approximate, timeout, число прогонов и допустимость найденного решения.

[MoveIt: Planning Scene](#) — Collision checking, Constraints, State validity. В задаче 4 включи самостолкновения и столкновения с окружением; проверь изменившуюся сцену перед исполнением.

[Nav2: Detailed Behavior Tree Walkthrough](#) — Navigation Subtree и Recovery Subtree. К задачам 7–8: минимальная цепочка планирования, следования и восстановления; XML и плагины сверяй с Jazzy.

MDP/POMDP пока изучаются только на уровне различий состояния и наблюдения. После чтения объясни, почему точный путь в модели может оказаться неисполнимым, а отказ по времени должен оставаться видимым результатом сравнительных испытаний.

Термины и определения

Конфигурация q . Набор независимых координат, определяющих геометрию положения робота; скорости обычно входят уже в расширенное состояние.

Пространство конфигураций (C-space). Пространство всех конфигураций робота; столкновение геометрических тел задаёт запрещённую область этого пространства.

Проекция корпуса (footprint). Проекция занимаемой роботом области на плоскость, используемая в планировании мобильной базы и проверке проходов.

Карта стоимости (costmap). Дискретное представление препятствий и штрафов движения; стоимость клетки не обязана быть вероятностью её занятости.

Геометрический путь $q(s)$. Последовательность конфигураций, параметризованная продвижением s ; время, скорость и ускорение ещё не определены.

Траектория $q(t)$. Движение, заданное во времени; позволяет вычислить скорости, ускорения и требования к исполнительным механизмам.

Dijkstra. Поиск кратчайших путей по неотрицательным стоимостям рёбер с последовательной обработкой минимальной текущей стоимости.

A*. Поиск, выбирающий узел по сумме накопленной стоимости g и эвристической оценки оставшегося пути h .

Допустимая эвристика. Оценка оставшейся стоимости, не превышающая истинный оптимум; гарантия поиска также зависит от реализации и условий графа.

PRM. Метод построения дорожной карты из допустимых случайных конфигураций и проверенных соединений между соседними выборками.

RRT. Дерево поиска, расширяемое к выборкам пространства; базовый алгоритм нацелен на нахождение пути, а не его оптимальность.

Вероятностная полнота. Свойство, при котором вероятность найти существующее решение стремится к единице при росте ресурса и выполнении предположений.

Проверка столкновений. Проверка пересечения геометрии робота с собой и миром; редкая проверка точек может пропустить столкновение между ними.

Допустимость состояния. Выполнение заданных ограничений конфигурации, сцены и задачи; не подтверждает допустимость всей соединяющей траектории.

Временная параметризация. Назначение зависимости продвижения по пути от времени с учётом ограничений скорости, ускорения и других условий.

Рывок (jerk). Производная ускорения по времени; выражается в м/с^3 либо рад/с^3 и характеризует резкость изменения воздействия.

Дерево поведения (BT). Иерархическая структура исполнения действий и условий, возвращающих статусы успеха, отказа или продолжения работы.

Вызов узла дерева (tick). Сигнал очередной проверки узла; частота вызовов определяет своевременность проверки условий и отмены действий.

FSM. Конечный автомат с явными состояниями и переходами по событиям или условиям; определяет допустимый порядок поведения.

Восстановление (recovery). Ограниченная процедура восстановления после обнаруженного отказа, имеющая условие завершения, лимит повторов и путь безопасной остановки.

Что купить и установить в этом месяце

Используй имеющееся оборудование

MacBook Pro M4, UTM с Ubuntu 24.04 ARM64, ROS 2 Jazzy, Gazebo Harmonic, карты, сцены и записи M14–M15. Для двух учебных стендов нужны модели мобильной базы и манипулятора в симуляции. Новых обязательных аппаратных покупок нет; физическую руку, колёсную базу или дополнительные приводы не заказывай.

Установи сейчас

В существующей гостевой Ubuntu установи Nav2 navigation2/nav2_bringup, MoveIt 2 и OMPL с зависимостями для используемой версии Jazzy. [MoveIt: Binary Install](#) указывает ros-jazzy-moveit для Ubuntu 24.04; [Nav2: официальный репозиторий](#) — отправная точка для установки пакетов и совместимого bringup. Зафиксируй пакетные версии и доступность arm64, не переноси команды Rolling без сверки.

MoveIt 2 устанавливается уже в M16, поскольку здесь выполняется лаборатория сцены манипулятора; в M17 этот стек используется повторно. OMPL сначала используй из зависимостей MoveIt, отдельную сборку C++ для сравнительных испытаний добавляй только для требуемых задач метрик. Привязки Python не являются обязательной зависимостью собственного A*/Dijkstra.

BehaviorTree.CPP используй в версии, согласованной с Nav2; [Reactive and Asynchronous behaviors](#) поясняет Sequence/ReactiveSequence и halt. GUI-редактор БТ необязателен. Сохрани один RMW, конфигурацию RViz и подготовленные демо-пакеты; не обновляй всё рабочее пространство ради новой страницы документации.

Проверь установку

1) В Nav2 отправь достижимую цель на небольшой карте, затем недостижимую; проверь результат действия action и отмену. 2) В MoveIt загрузи готовую модель, стол и коробку; получи путь и отклони столкновение. 3) Проверь доступность выбранных планировщиков OMPL, статусы exact/approximate и таймаут. 4) Запиши RUNNING, отмену и восстановление дерева, а также окончательное прекращение команды.

RViz и Gazebo в UTM проверь отдельным малым запуском: наличие пакета не доказывает работу графики. При проблемах сохрани консольные логи и пути для анализа; визуальный критерий остаётся открытым до настоящего просмотра. Для этих симуляционных задач USB не нужен, но повторно используемая физическая периферия требует прежней проверки.

Чтение и подготовка учитываются внутри 75 часов. Бортовой Pi/x86, GPU и комплект физической руки относятся к последующим условным этапам; после M17 отдельный OPT120 имеет собственные CAD/BOM и проверку одного пальца.

Основы движения: время вдоль пути

Обязательно: почему геометрии недостаточно

Одинаковый путь можно пройти медленно или резко. Столкновения задаются геометрией конфигураций, а скорости, ускорения и требуемые моменты зависят также от времени. Предпосылка: возьми один вращательный сустав ноги или пальца, q в радианах, и объясни производные $\dot{q}$ и $\ddot{q}$ с единицами рад/с и рад/с².

Две функции вместо одного списка точек

Путь $q(s)$, $0 \leq s \leq 1$, задаёт порядок конфигураций; монотонное $s(t)$ задаёт продвижение. По цепному правилу $\dot{q} = q'(s)\dot{s}$ и $\ddot{q} = q''(s)\dot{s}^2 + q'(s)\ddot{s}$. Первая часть ускорения связана с кривизной пути, вторая — с изменением темпа. Поэтому гладкость $q(s)$ не заменяет проверки временных производных.

Для прямого суставного пути $q(s) = q_0 + \Delta q \cdot s$ выбери $s(t) = 3\xi^2 - 2\xi^3$, $\xi = t/T$. Тогда $\dot{q} = (\Delta q/T)(6\xi - 6\xi^2)$, $\ddot{q} = (\Delta q/T^2)(6 - 12\xi)$. Скорость в начале и конце нулевая; ускорение там ненулевое и при переходе к покою изменяется скачком. Квинтический профиль прежней задачи 5 добавляет нулевые граничные ускорения.

Численный пример и выбор длительности

Пусть $q_0 = 0$, $\Delta q = 0,6$ рад, $T = 2$ с. Максимальная скорость в $t = 1$ с: $1,5\Delta q/T = 0,45$ рад/с. Максимум $|\ddot{q}|$ на концах равен $6\Delta q/T^2 = 0,90$ рад/с². При учебных пределах 0,30 рад/с и 0,60 рад/с² этот профиль слишком быстр.

Ограничение скорости даёт $T \geq 1,5 \cdot 0,6 / 0,30 = 3$ с; ограничение ускорения — $T \geq \sqrt{(6 \cdot 0,6 / 0,60)} = \sqrt{6} \approx 2,45$ с. Значит, для этого кубического профиля требуется $T \geq 3$ с. При $T = 3$ с максимумы 0,30 рад/с и 0,40 рад/с²; для запаса выбери большее T . Это вывод для выбранной формы, а не глобально минимальное время любого движения.

Скорость стопы равна $J_p(q)\dot{q}$ и меняется с конфигурацией; один суставной предел не задаёт постоянной скорости стопы. Кроме того, момент удержания веса не исчезает при медленном движении. После проверки ограничений скорости и ускорения проверь динамику, контакты и геометрию; расчёт момента не задаёт режим управления позиционному сервоприводу.

Два упражнения в задачах 5-6

1. Сохрани $q, \dot{q}, \ddot{q}$ для $T = 2, 3, 4$ с в trajectory.csv. Сравни аналитические экстремумы с плотной сеткой и численными производными; проверь $q(0), q(T)$, нулевые скорости и отказ при $T \leq 0$. Затем тем же валидатором проверь квинтический профиль задачи 5: коэффициенты максимума у него иные.

2. Возьми путь руки возле коробки: монотонное изменение только временной параметризации сохраняет геометрию пути, а сглаживание пути по опорным точкам может его изменить. Покажи обе версии и повторную проверку столкновений; приложи один корректный отказ и график нарушенного ограничения.

Учёт времени: 1 час из 41–42 на вывод; по 1 часу из 51–52 и 53–54 на численный пример и валидатор. Проверка геометрии входит в прежние 57–58. Бюджет остаётся 75 часов, дополнительный стенд не нужен.

Чтение: Lynch/Park, Modern Robotics §9.1–9.2, часть 1 — путь и время; часть 2 — кубическое и квинтическое масштабирование времени и граничные условия.